

Лекция 8. Отладка программ

- Виды ошибок
- Тестирование
- Трассировка
- Отладочная печать

Отладка

- Все сложнее, чем кажется
- Все тянется дольше, чем можно ожидать
- Если что-то может испортиться, оно обязательно испортится

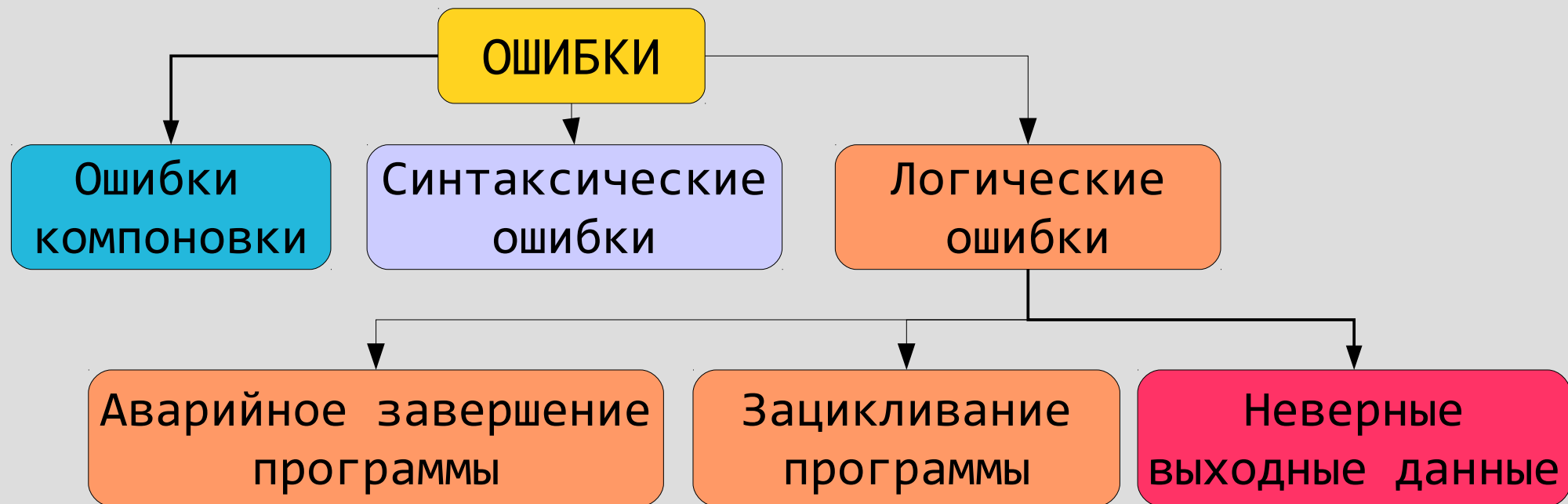
Законы Мэрфи

Отладка

- Каждая последняя обнаруженная ошибка является предпоследней
- Когда программа полностью отлажена она становится ненужной
- Чем опытней программист, тем трудноуловимей становятся его ошибки
- Тестирование и отладка занимает 60-75% времени от разработки программы

Аксиомы отладки

Виды ошибок и методы их локализации



Методы	Сообщение компоновщика об ошибке	Сообщение компилятора об ошибке	Unit-тестирование
			Трассировка
			Отладочная печать
	Визуальный анализ кода		
	Комментирование части кода		

Виды ошибок

- Ошибки **компоновки** — обычно возникают, когда функция заявлена (имеется прототип), но не реализована (отсутствует тело)
- **Синтаксические** ошибки — это ошибки, возникающие при нарушении синтаксиса языка
- **Семантические** (логические) ошибки — это ошибки, возникающие когда программа выполняет то, что вы написали, а не то, что вы хотели

Принципы отладки синтаксических ошибок

- О синтаксических ошибках **сообщает компилятор**, нередко с указанием причины ошибки и ее местонахождения (т.е. строки, в которой находится ошибка)
- **Большинство** синтаксических ошибок сравнительно **легко** найти и исправить
- **Некоторые** ошибки "**трудноуловимы**" для компилятора и он выдает сообщение сразу о нескольких ошибках. В этом случае следует исправить **самую первую** ошибку и, возможно, остальные ошибки **исчезнут** сами по себе

Рекомендации по отладке синтаксических ошибок

- Начинаяте отладку с **самой первой** ошибки
- **Внимательно читайте** сообщения компилятора, т.к. для большинства ошибок выдаются правдоподобные описания
- Если вы не видите ошибки в строке, на которую указывает компилятор, то проанализируйте **соседние** строки, расположенные сверху и снизу, возможно, ошибка там

Рекомендации по отладке синтаксических ошибок

- Если ошибка, на которую указывает компилятор, все равно **не видна**, то можно поступить следующим образом:
 - **пропустить** ошибку, возможно, это следствие другой ошибки
 - **закомментировать** часть кода, который компилятор считает ошибочным

Рекомендации по отладке синтаксических ошибок

- Если **после комментирования** кода сообщение об ошибке **не появляется**, значит ошибка все таки присутствует в закоментированном коде и его нужно внимательно изучить визуально
- Если **после комментирования** кода ошибка **осталась**, значит причина ошибки вне закоментированного кода

Логические ошибки

- Логические ошибки проявляются в виде:
 - аварийного завершения работы ("вылета") программы
 - "зависания" ("зацикливания") программы
 - неверных выходных данных
- Логические ошибки значительно труднее обнаружить и отладить, т.к. они не обнаруживаются компилятором и для их устранения требуется понимание решаемой задачи

Аварийное завершение программы

- Аварийное завершение программы обычно происходит при **некорректной** работе с **памятью** или выполнении **некорректных операций**
- Наиболее часто встречающиеся причины:
 - запись данных в ячейки памяти, которые не зарезервированы компилятором (например, при выходе за пределы массива)
 - обращение к неинициализированной переменной
 - деление на ноль

Принципы отладки аварийного завершения программы

- После аварийного завершения программы осуществляется переход в отладочный режим среды разработки
- При этом выдается сообщение об ошибке и стрелкой  выделяется строка, в которой произошла ошибка
- В случае обращения к неинициализированной переменной или делении на ноль ошибка легко локализуется и исправляется

Принципы отладки аварийного завершения программы

- Если ошибка вызвана обращением к незарезервированному участку памяти, то ошибка может возникнуть в произвольном месте программы. Чаще всего, это происходит в конце программы при освобождении памяти
- В этом случае можно применить отладочную печать (см. далее) чтобы узнать, к каким элементам массива обращалась программа в течение своей работы и был ли выход за пределы массива

Логические ошибки, приводящие к неверным выходным данным

- **Тестирование** — это процесс, посредством которого проверяется правильность программы
- **Цель** тестирования — показать, что программа работает на всех входных данных
- **Отладка** — это процесс исправления ошибок в программе
- Тестирование выполняется на **наборе тестов**, т.е. проверочных исходных данных и соответствующих им наборам эталонных реакций

Требования к набору тестов

- Для исчерпывающего тестирования программы необходимо чтобы тесты позволяли проверить:
 - **каждую** из **ветвей** алгоритма
 - **граничные** условия (например, переход по условию $x > 10$ должен проверяться для значений больших, меньших и равных 10)
 - реакцию программы на **ошибочных** исходных данных

Рекомендации по тестированию

- В арифметических выражениях необходимо учесть особые случаи: деление на ноль, квадратный корень из отрицательного значения и т.д.
- В алгоритмических структурах «альтернатива» или «выбор» необходимо проверить все пути, задаваемые этой структурой (даже если они ничего не делают)
- Любой цикл должен проверяться на тесте, когда его тело выполняется не менее 3-х раз

Рекомендации по тестированию

- Дополнительно параметрический цикл и цикл с предусловием должны проверяться на тесте, когда тело цикла не выполняется ни разу
- Цикл с постусловием должен проверяться на тесте, когда тело цикла выполняется однократно

Рекомендации по тестированию

- При тестировании алгоритмических структур «альтернатива» или «выбор», находящихся внутри цикла, необходимо проверить каждый путь развилки в трех ситуациях:
 - на первом шаге цикла
 - на последнем шаге цикла
 - на любом промежуточном шаге цикла

Методы отладки

- Главная задача отладки — получить представление о том, какие действия и в каком порядке выполняются и как при этом изменяются переменные
- **Трассировка** программы - **пошаговое** выполнение программы в среде разработки
- **Отладочная печать** состоит в выводе на экран сообщений о прохождении контрольных точек и значений интересующих переменных
- **Unit-тесты** — тесты, проверяющие работу отдельных функций программы

Задача

Отладить программу, удаляющую в массиве все элементы до первого вхождения минимального элемента массива

Задача

```
1 /* Удаляет в массиве все элементы до первого
2    вхождения минимального элемента массива */
3
4 #include "stdfix.h"
5 #include "testing.h"
6
7
8 void deleteElement(int mass[], int size,
9                   int index)
10 {
11     for(int i = index; i < size; i++)
12         mass[i-1] = mass[i];
13 }
```

Задача

```
15 int mass[5] = { 1, 2, 3, 0, 5 };
16 int indexMin = 0;
17 int n = 5;
18
19 for(int i = 0; i < n; i++)
20     if(mass[i] <= mass[indexMin])
21         indexMin = i;
22
23 for(int i = 0; i < indexMin; i++)
24 {
25     deleteElement(mass, n, i);
26     n--;
27 }
28
29 printMass(mass, n);
30 WAIT_ANY_KEY
```

Задание

Составить набор тестовых примеров
для проверки этой задачи

Тестовые примеры

- Тест 1: Минимальный элемент в середине массива

1 2 3 0 5 => 0 5

- Тест 2: Минимальный элемент является первым

0 1 2 3 5 => 0 1 2 3 5

- Тест 3: Минимальный элемент является последним

1 2 3 5 0 => 0

- Тест 4: Несколько минимальных элементов

1 0 3 0 5 => 0 3 0 5

- Тест 5: Все элементы массива одинаковы

1 1 1 1 1 => 1 1 1 1 1

Запуск программы и ее аварийное завершение

```
15 int mass[5] = { 1, 0, 2, 5, 4 }; // исходный мас
16 int indexMin; // индекс миним
17 int n = 5; // реальное кол
18 // Находим минимальный элемент
19 for(int i = 0; i < n; i++)
20     if(mass[i] < mass[indexMin]) // Запоминаем текущий элеме
21         indexMin = i; //если он меньше текущего m
22 // Удаляем последовательность элементов
23 for(int i = 0; i < indexMin; i++)
24
25
26
27
28
29
30
31 }
32 //
33 void
34 {
25 for(int i = indexMin; i < size; i++)
```

Microsoft Visual Studio

 Run-Time Check Failure #3 - The variable 'indexMin' is being used without being defined.

Break Continue Ignore

Отладка вылета программы

- По сообщению и выделенной желтой стрелкой строке ясно, что проблема в отсутствии инициализации переменной `indexMin`
- Для решения проблемы необходимо присвоить ей начальное значение:

```
16      int indexMin = 0;
```

Проверка программы на тесте

- Исходный массив: 1 2 3 0 5
- Ожидаемый результат: 0 5
- Реальный результат: 3 5

Логическая ошибка!

Отладка сложной программы

- Если выполнение программы состоит из нескольких последовательных этапов, то до определения причины ошибки следует **локализовать** ее: определить, на каком из этапов выполнения она происходит
- Это можно сделать, расставив точки останова в начале/конце каждого этапа и проверив выходные данные каждого этапа на корректность

Задание

Отметьте точки останова в начале/конце этапов программы

Определите назначение этапов и выходные данные каждого этапа

Задание

```
15 int mass[5] = { 1, 2, 3, 0, 5 };
16 int indexMin = 0;
17 int n = 5;
18
19 for(int i = 0; i < n; i++)
20     if(mass[i] <= mass[indexMin])
21         indexMin = i;
22
23 for(int i = 0; i < indexMin; i++)
24 {
25     deleteElement(mass, n, i);
26     n--;
27 }
28
29 printMass(mass, n);
30 WAIT ANY KEY
```

Точки останова в начале/конце этапов

```
15 int mass[5] = { 1, 2, 3, 0, 5 };
16 int indexMin = 0;
17 int n = 5;
18
19 for(int i = 0; i < n; i++)
20     if(mass[i] <= mass[indexMin])
21         indexMin = i;
22
23 for(int i = 0; i < indexMin; i++)
24 {
25     deleteElement(mass, n, i);
26     n--;
27 }
28
29 printMass(mass, n);
30 WAIT ANY KEY
```

Точки останова в начале/конце этапов

- Точка останова в строке 19 позволяет проанализировать корректность **ИСХОДНЫХ ДАННЫХ**, т.е. начальные значения переменных **mass**, **n**
- Точка останова в строке 23 позволяет увидеть результаты выполнения первого этапа — **поиска первого минимального элемента**, т.е. анализируем значение переменной **indexMin**

Точки останова в начале/конце этапов

- Точка останова в строке 29 позволяет увидеть результаты выполнения второго этапа — **удаление последовательности элементов** до первого минимального элемента, т.е. анализируем значения переменных **mass**, **n**
- Точка останова в строке 30 позволяет увидеть результаты выполнения последнего этапа — **печать массива на экран**, т.е. следует просмотреть, что **выводится на экран**

Результаты поэтапной трассировки

Переменная	Ожидаемый результат	Реальный результат
------------	---------------------	--------------------

Этап 1: Инициализация исходных данных

mass	{ 1, 2, 3, 0, 5 }	{ 1, 2, 3, 0, 5 }
n	5	5

Этап 2: Определение индекса минимального элемента

indexMin	3	3
----------	---	---

Этап 3: Удаление элементов до минимального

mass	{ 0, 5 }	{ 3, 5 }
------	----------	----------

Этап 4: Печать массива

Не выполняется, т.к. ранее обнаружен ошибка

Отладка этапов с функциями

- Этап, на котором обнаружена ошибка, содержит функцию
- Для локализации ошибки необходимо проверить, правильно ли работает функция
- Для проверки правильности работы функции используются **unit-тесты**

Unit-тесты

- Unit-тесты применяются для проверки работоспособности функций
- Unit-тест представляет собой программу, которая вызывает тестируемую функцию с определенными исходными данными, а затем проверяет правильность выходных данных на эталонных значениях
- Unit-тесты позволяют целенаправленно тестировать некоторую функцию отдельно от ее окружения

Unit-тестирование - из учебной практики

- Написание unit-тестов — работа, которая кажется нудной и не очень нужной
- Никто не любит писать unit-тесты
- Студенты готовы на все: отлаживать программу неделями, не сдавать вовремя лабораторные работы — лишь бы не писать unit-тесты

Задание

Составьте набор тестовых примеров для функции

```
void deleteElement(int mass[], int size, int index)
```

где

`mass` – массив, из которого удаляется элемент

`size` – размер массива (до удаления)

`index` – индекс удаляемого элемента

Тестовые примеры

- Исходный массив

size = 5 { 1 0 2 5 4 }

- Тест 1: Удаляется первый элемент

index = 0 => { 0 2 5 4 }

- Тест 2: Удаляется средний элемент

index = 1 => { 1 2 5 4 }

- Тест 3: Удаляется последний элемент

index = 4 => { 1 0 2 5 }

- Тест 4: Удаляется единственный элемент

size = 1 { 1 }

39 index = 0 => { }

Задание

Напишите unit-тесты для функции.

Код тестирования должен вызывать функцию с исходными данными и печатать для сравнения ожидаемый и реальный результаты выполнения

```
void deleteElement(int mass[], int size, int index)
```

где

mass – массив, из которого удаляется элемент

size – размер массива (до удаления)

index – индекс удаляемого элемента

Unit-тесты

// Входные данные

```
int input_size[4] = {5, 5, 5, 1};
```

```
int input_mass[4][5] = {  
    {1, 0, 2, 5, 4 },  
    {1, 0, 2, 5, 4 },  
    {1, 0, 2, 5, 4 },  
    {1} };
```

```
int input_index[4] = {0, 1, 4, 0};
```

// Ожидаемые результаты

```
int expected_results[4][4] = {  
    {0, 2, 5, 4},  
    {1, 2, 5, 4},  
    {1, 0, 2, 5},  
    {} };
```

Unit-тесты

```
// Запускаем тесты
for(int i = 0; i < 4; i++)
{
    printf("\nTest: %d\n", i+1);
    printf("Starting array: ");
    printMass(testing_data[0], input_size[i]);
    printf("\nIndex of deleted element: %d", del_index[i]);
    printf("\nExpected result: ");
    printMass(expected_results[i], input_size[i]-1);

    deleteElement(input_mass[i], input_size[i],
                  del_index[i]);

    printf("\nReal result:      ");
    printMass(testing_data[i], input_size[i]-1);
}
```

Результаты unit-тестирования

Test: 1

Starting array: 1, 0, 2, 5, 4

Index of deleted element: 0

Expected result: 0, 2, 5, 4

Real result: 0, 2, 5, 4

Test: 2

Starting array: 1, 0, 2, 5, 4

Index of deleted element: 1

Expected result: 1, 2, 5, 4

Real result: 0, 2, 5, 4

Результаты unit-тестирования

Test: 3

Starting array: 1, 0, 2, 5, 4

Index of deleted element: 4

Expected result: 1, 0, 2, 5

Real result: 1, 0, 2, 4

Test: 4

Starting array: 1

Index of deleted element: 0

Expected result: empty array

Real result: empty array

ОШИБКА: Run-Time Check Failure #2....

Отладочная печать

- Отладочная печать применяется, если:
 - переменная имеет сложную структуру и ее значение неудобно просматривать во время останова программы
 - необходимо получить быстрый «снимок» состояний переменной в нескольких контрольных точках, чтобы определить, в какой части программы произошла ошибка
 - программа не может быть остановлена, т.к. она взаимодействует в реальном времени с другой программой или физическим устройством

Задание

Какую информацию необходимо вывести на печать, чтобы понять работу функции

```
void deleteElement(int mass[], int size, int index)
{
    for(int i = index; i < size; i++)
        mass[i-1] = mass[i];
}
```

Отладочная печать

- В данном случае функция состоит из цикла, **перемещающего** элементы массива
- Поэтому для понимания его работы нам важно знать **индексы** перемещаемых элементов: **$i-1$** и **i**
- **Значения** этих элементов для понимания того, правильно ли работает **перемещение**, **не важны**

Задание

Вставьте отладочную печать для проверки работы функции

```
void deleteElement(int mass[], int size, int index)
{
    for(int i = index; i < size; i++)
        mass[i-1] = mass[i];
}
```


Отладочная печать

```
void deleteElement(int mass[], int size, int index)
{
    for(int i = index; i < size; i++)
    {
        printf("\nmass[%d] <= mass[%d]", i-1, i);
        mass[i-1] = mass[i];
    }
}
```

Отладочная печать

Test: 2

Starting array: 1, 0, 2, 5, 4

Index of deleted element: 1

Expected result: 1, 2, 5, 4

```
mass[0] <= mass[1]
```

```
mass[1] <= mass[2]
```

```
mass[2] <= mass[3]
```

```
mass[3] <= mass[4]
```

Real result: 0, 2, 5, 4

Вывод: перестановка начинается на один элемент
раньше чем положено

Задание

Исправьте ошибку в функции

```
void deleteElement(int mass[], int size, int index)
{
    for(int i = index; i < size; i++)
        mass[i-1] = mass[i];
}
```

Исправленная ошибка

```
void deleteElement(int mass[], int size, int index)
{
    for(int i = index; i < size; i++)
        mass[i-1] = mass[i];
}
```

```
void deleteElement(int mass[], int size, int index)
{
    for(int i = index + 1; i < size; i++)
        mass[i-1] = mass[i];
}
```

Результаты повторного unit-тестирования

Test: 1

Starting array: 1, 0, 2, 5, 4

Index of deleted element: 0

Expected result: 0, 2, 5, 4

Real result: 0, 2, 5, 4

Test: 2

Starting array: 1, 0, 2, 5, 4

Index of deleted element: 1

Expected result: 1, 2, 5, 4

Real result: 1, 2, 5, 4

Результаты повторного unit-тестирования

Test: 3

Starting array: 1, 0, 2, 5, 4

Index of deleted element: 4

Expected result: 1, 0, 2, 5

Real result: 1, 0, 2, 5

Test: 4

Starting array: 1

Index of deleted element: 0

Expected result: empty array

Real result: empty array

Функция работает корректно !

Повторная проверка программы на тесте

- Исходный массив: 1 2 3 0 5
- Ожидаемый результат: 0 5
- Реальный результат: 2 0

Логическая ошибка!

Повторная трассировка программы

```
15 int mass[5] = { 1, 2, 3, 0, 5 };
16 int indexMin = 0;
17 int n = 5;
18
19 for(int i = 0; i < n; i++)
20     if(mass[i] <= mass[indexMin])
21         indexMin = i;
22
23 for(int i = 0; i < indexMin; i++)
24 {
25     deleteElement(mass, n, i);
26     n--;
27 }
28
29 printMass(mass, n);
30 WAIT_ANY_KEY
```


Результаты повторной поэтапной трассировки

Переменная	Ожидаемый результат	Реальный результат
------------	---------------------	--------------------

Этап 1: Инициализация исходных данных

mass	{ 1, 2, 3, 0, 5 }	{ 1, 2, 3, 0, 5 }
n	5	5

Этап 2: Определение индекса минимального элемента

indexMin	3	3
----------	---	---

Этап 3: Удаление элементов до минимального

mass	{ 0, 5 }	{ 2, 0 }
------	----------	----------

Этап 4: Печать массива

Не выполняется, т.к. ранее обнаружен ошибка

Задание

Определите, что необходимо вывести на печать для понимания работы следующего кода

```
for(int i = 0; i < indexMin; i++)  
{  
    deleteElement(mass, n, i);  
    n--;  
}
```

Отладочная печать

- В данном случае, в цикле **удаляется** последовательность элементов массива
- Поэтому для понимания работы цикла нам важно знать **индексы** удаляемых элементов, т.е. ***i***
- **Желательно** также видеть сам **массив** после удаления очередного элемента (или с выделением удаляемого элемента цветом)

Задание

Вставьте отладочную печать для проверки работы этапа (для печати массива можно использовать функцию `printMass(int array[], int size)`)

```
for(int i = 0; i < indexMin; i++)  
{  
    deleteElement(mass, n, i);  
    n--;  
}
```

Отладочная печать

```
printMass(mass, n);  
for(int i = 0; i < indexMin; i++)  
{  
    deleteElement(mass, n, i);  
    n--;  
    printf("\ni = %d\n", i);  
    printMass(mass, n);  
}
```

Отладочная печать

1, 2, 3, 0, 5

i = 0

2, 3, 0, 5

i = 1

2, 0, 5

i = 2

2, 0

Вывод: на втором шаге **должен удаляться первый элемент**, а **реально** удаляется **второй элемент**. На третьем шаге снова должен удаляться первый элемент, а удаляется третий

Задание

Исправьте обнаруженную ошибку

```
for(int i = 0; i < indexMin; i++)  
{  
    deleteElement(mass, n, i);  
    n--;  
}
```

Ошибка исправлена

```
for(int i = 0; i < indexMin; i++)  
{  
    deleteElement(mass, n, i);  
    n--;  
}
```

```
for(int i = 0; i < indexMin; i++)  
{  
    deleteElement(mass, n, 0);  
    n--;  
}
```


Проверка программы на тесте

- Исходный массив: 1 2 3 0 5
- Ожидаемый результат: 0 5
- Реальный результат: 0 5

Все правильно!

Вопрос

Можем ли мы быть уверены в работоспособности программы?

Тестовые примеры

- Тест 2: Минимальный элемент является первым

0 1 2 3 5 => 0 1 2 3 5

Реально: 0 1 2 3 5

- Тест 3: Минимальный элемент является последним

1 2 3 5 0 => 0

Реально: 0

- Тест 4: Несколько минимальных элементов

1 0 3 0 5 => 0 3 0 5

•Реально: 0 5

Снова логическая ошибка!

Повторная трассировка программы

```
15 int mass[5] = { 1, 0, 3, 0, 5 };
16 int indexMin = 0;
17 int n = 5;
18
19 for(int i = 0; i < n; i++)
20     if(mass[i] <= mass[indexMin])
21         indexMin = i;
22
23 for(int i = 0; i < indexMin; i++)
24 {
25     deleteElement(mass, n, i);
26     n--;
27 }
28
29 printMass(mass, n);
30 WAIT_ANY_KEY
```

Результаты повторной поэтапной трассировки

Переменная	Ожидаемый результат	Реальный результат
------------	---------------------	--------------------

Этап 1: Инициализация исходных данных

mass	{ 1, 0, 3, 0, 5 }	{ 1, 0, 3, 0, 5 }
n	5	5

Этап 2: Определение индекса минимального элемента

indexMin	1	3
----------	---	---

Этап 3: Удаление элементов до минимального

Не выполняется, т.к. ранее обнаружен ошибка

Этап 4: Печать массива

Не выполняется, т.к. ранее обнаружен ошибка

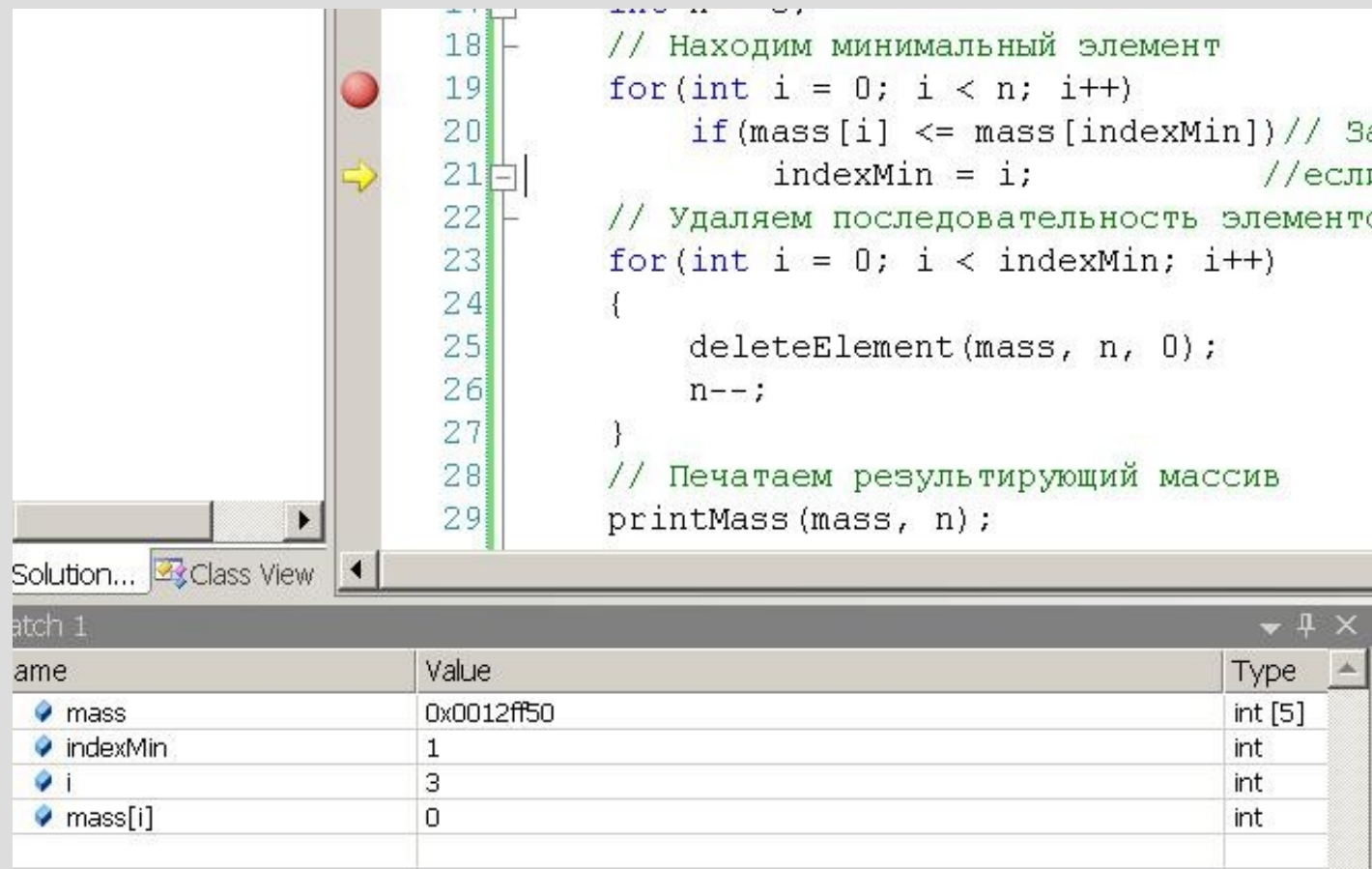
Пошаговое выполнение программы

- **Пошаговое выполнение программы** выполняется, если анализируемый переменные имеют простую структуру и кол-во выполняемых операторов (с учетом повторения) достаточно небольшое
- **Единицей** (шагом) выполнения программы при трассировке является **строка**, а не оператор
- Место начала трассировки задается с помощью **точки останова**
- Если точка останова не задана, то трассировка начинается с первой строки программы

Команды пошагового выполнения программы

- **Step Into** (клавиша **F11**) — последовательное, строка за строкой, выполнение программного кода программы с "заходом" в тело функции
- **Step Over** (клавиша **F10**) — аналогична команде Step Into, но функция выполняется за один шаг
- **Step Out** (клавиша **Shift+F11**) — выход из функции — все оставшиеся строки тела функции выполняются за один шаг — чаще всего, используется после случайного выполнения команды Step Into

Трассировка этапа 1



```
18 // Находим минимальный элемент
19 for(int i = 0; i < n; i++)
20     if(mass[i] <= mass[indexMin])// За
21         indexMin = i;           //если
22 // Удаляем последовательность элементов
23 for(int i = 0; i < indexMin; i++)
24 {
25     deleteElement(mass, n, 0);
26     n--;
27 }
28 // Печатаем результирующий массив
29 printMass(mass, n);
```

Name	Value	Type
mass	0x0012ff50	int [5]
indexMin	1	int
i	3	int
mass[i]	0	int

Программа выполняет строку 21 и для второго минимального элемента, хотя должна выполняться только для первого минимального элемента

Задание

Исправьте обнаруженную ошибку

```
for(int i = 0; i < n; i++)  
    if(mass[i] <= mass[indexMin])  
        indexMin = i;
```

Ошибка исправлена

```
for(int i = 0; i < n; i++)  
    if(mass[i] <= mass[indexMin])  
        indexMin = i;
```

```
for(int i = 0; i < n; i++)  
    if(mass[i] < mass[indexMin])  
        indexMin = i;
```

Повторная проверка на тестовых примерах

- Тест 1: Один минимальный элемент

1 2 3 0 5 => 0 5

•Реально: 0 5

- Тест 2: Минимальный элемент является первым

0 1 2 3 5 => 0 1 2 3 5

Реально: 0 1 2 3 5

- Тест 3: Минимальный элемент является последним

1 2 3 5 0 => 0

Реально: 0

Повторная проверка на тестовых примерах

- Тест 4: Несколько минимальных элементов

1 0 3 0 5 => 0 3 0 5

Реально: 0 3 0 5

- Тест 5: Все элементы массива одинаковы

1 1 1 1 1 => 1 1 1 1 1

Реально: 1 1 1 1 1

Все работает!

Рекомендации по трассировке ветвления

- Если обнаружена ошибочная альтернатива (выбор), то с помощью пошагового выполнения (команды Step Into или Step Over) необходимо определить в какую из веток переходит программа
- Если неверно выбирается ветка альтернативы, то ошибка кроется в условии
- Если условие верно, то ошибка локализуется внутри ветки последовательным способом, как было описано выше

Рекомендации по трассировке цикла

- Прежде, чем трассировать цикл необходимо убедиться в работоспособности его тела
- Трассировка тела цикла выполняется последовательным способом
- Если тело цикла выполняется верно, то трассируется весь цикл; при этом тело цикла выполняется за один шаг
- Для трассировки тела цикла за один шаг точка останова ставится на последнюю строку тела цикла, которая затем выполняется с помощью команды Step Into или Step Over

Рекомендации по трассировке функции

- Сначала нужно проверить работоспособность функции в целом, для чего используется команда Step Over и unit-тесты
- Если выходные/выводимые данные функции неверны, то выполняется трассировка самой функции
- Трассировка функции начинается с проверки значений входных и выходных данных

Рекомендации по трассировке функции

- С этой целью ставятся 2 точки останова:
 - на первую строку тела функции
 - на строку, содержащую оператор `return`, или последнюю строку тела функции
- Затем анализируются значения аргументов функции и возвращаемого выражения
- Если входные данные верны, а выходные - нет, то последовательным способом трассируется тело функции

"Зависание" программы

- Зависание программы обычно говорит о наличии в ней цикла, который не заканчивается, или "**бесконечного**" цикла
- Причинами бесконечного цикла являются:
 - неверное условие цикла — оно всегда истинное
 - не изменяется переменная цикла
- **Переменная цикла** — это переменная, которая участвует в условии цикла

Принципы отладки "зависания" программы

- Локализовать бесконечный цикл можно путем трассировки или отладочной печати
- Признаком бесконечного цикла является ситуация, когда одна контрольная точка выполняется, а следующая за ней - нет (считается, что контрольные точки не разделены ветвлением)
- В этом случае бесконечный цикл находится между указанными контрольными точками

Рекомендации по отладке "бесконечного" цикла

- Проанализируйте условие цикла и, по возможности, откорректируйте его
- Если условие цикла кажется верным, то посмотрите, как меняется переменная цикла
- Если значение переменной не меняется вовсе или меняется неверно, то исправьте ошибку
- Если цикл не "разблокировался", то на каждой итерации проанализируйте условие цикла еще раз и исправьте ошибку

Это надо помнить!

Главное отличие профессионала от любителя —
умение (и необходимость) качественно решить
задачу в ограниченные сроки

Это надо помнить!

- **Пренебрежение** рутинными методами отладки — unit-тестированием, трассировкой, отладочной печатью — может **увеличить время отладки** на несколько часов или дней, а в сложных программах недель или даже месяцев
- **Если** вы **смогли** догадаться о причине ошибки просто глядя на код программы **за 15 минут — хорошо, если нет — начинайте систематическую отладку**

Это надо помнить!

- Составляя набор тестов старайтесь предусмотреть все случаи работы программы, особенно необычные и более трудные. Лишний тест не приведет к ошибке, в то время как недостаточное количество тестов может дать вам пропустить ошибку. Нет хуже той ошибки, что пропустили вы и заметил начальник
- После каждого изменения программу следует проверять на полном наборе тестов, чтобы быть уверенным, что ничего не было испорчено

Это надо помнить!

- При отладке программы, состоящей из нескольких **последовательных этапов** следует применять **поэтапную трассировку**, установив точки останова в начале каждого этапа, и нажимая **Continue** (F5, ►) переходить к следующему этапу, проверяя корректность данных в конце каждого этапа

Это надо помнить!

- Если отлаживаемая программа содержит нетривиальные **функции**, то работоспособность функций следует проверить с помощью **unit-тестов** — фрагмента программы, вызывающего функцию с тестовыми данными и проверяющего правильность результата (либо печатающего на экран ожидаемый и реальный результаты для сравнения)

Это надо помнить!

- **Не стирайте** написанные unit-тесты — они могут пригодиться вам, если вы будете изменять код функции. Просто уберите из программы их вызов.
- Чтобы проверить, хорошо ли вы понимаете, что должна делать функция, напишите unit-тесты **до** написания кода самой функции

Это надо помнить!

- Для получения снимка изменения ключевых переменных в ходе цикла, либо проверки содержимого массивов удобно использовать **отладочную печать**
- **Трассировку** удобно использовать в случае, если просматриваемый участок повторяется небольшое количество раз, а переменные, требующие проверки являются скалярными (хранят одно значение)